



Neural Networks

The Building Blocks of Deep Learning

30-Day AI Course • Day 10 of 30

Perceptron • Activation Functions • Backpropagation • Gradient Descent

Awet G. Nway

Founder, AYE Tech Hub • Engineering Education & Services

www.ayetechub.com

AYE Tech Hub • 30-Day AI Course



Welcome to Day 10

Welcome back to the AYE Tech Hub 30-Day AI Course! In the first nine days you learned what AI is, how machine learning models work, and how classical algorithms like linear regression and decision trees make predictions from data. Today we cross the bridge into deep learning — and it all starts with one beautifully simple idea: the artificial neuron.

Neural networks are the engine behind almost every modern AI breakthrough — image recognition, machine translation, ChatGPT, AlphaFold, self-driving cars. Despite the hype, the math behind them is approachable. By the end of today you will understand the four ideas every deep-learning practitioner builds on for the rest of their career.

What you will learn today

#	Topic	Key takeaway
1	From biological to artificial neurons	Inputs × weights + bias → activation.
2	Feedforward networks	Stacking neurons into layers makes them powerful.
3	Activation functions	Sigmoid, ReLU, Tanh — and why ReLU won.
4	How neural networks learn	Forward pass, loss, backprop, gradient descent.
5	Practical exercise & quiz	Compute a perceptron output by hand.

i INFO A quick recap of Day 9

Yesterday we wrapped up classical machine learning with decision trees and ensemble methods (random forests, gradient boosting). Those models still dominate tabular data competitions. Today we shift to the family of models that dominates everything else — images, audio, text, and beyond.

1 From Biological to Artificial Neurons

In 1943, neurophysiologist Warren McCulloch and logician Walter Pitts published the first mathematical model of a brain cell — what we now call an artificial neuron or perceptron. Their insight: a real neuron fires when the weighted sum of its inputs crosses a threshold. That single sentence is still the core of every neural network in 2026.

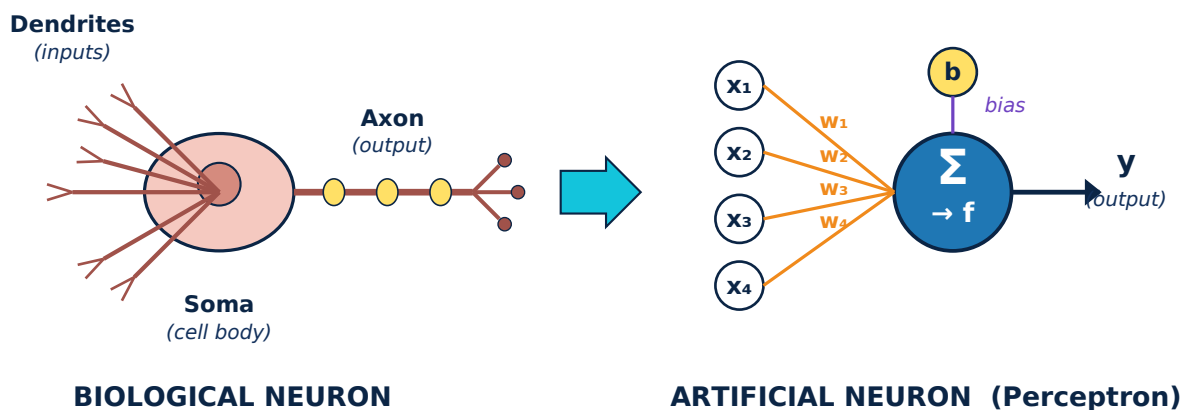


Figure 1.1 — A biological neuron (left) and its artificial counterpart (right)

The perceptron formula

$$y = f (w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n + b)$$

Each input x_i is multiplied by its own weight w_i , all products are summed, a bias b is added to shift the result, and the total is squashed by an activation function f . The output y is what the neuron "believes". Training a network means finding the values of w and b that make those beliefs match reality on labelled examples.

✓ TIP Why the bias term matters

Without a bias, every neuron's output passes through the origin — it can only learn patterns that go through $(0, 0)$. The bias term b shifts the activation curve left or right, just like the y-intercept of a line. Always include it; never set it to zero permanently.

2

Feedforward Networks: Stacking Neurons

A single perceptron can only learn linearly separable patterns — it cannot even solve the XOR problem. The breakthrough came in the 1980s when researchers stacked perceptrons into layers, with each layer feeding the next. This is the multi-layer perceptron (MLP), the simplest deep neural network.

FEEDFORWARD NEURAL NETWORK (Multi-Layer Perceptron)

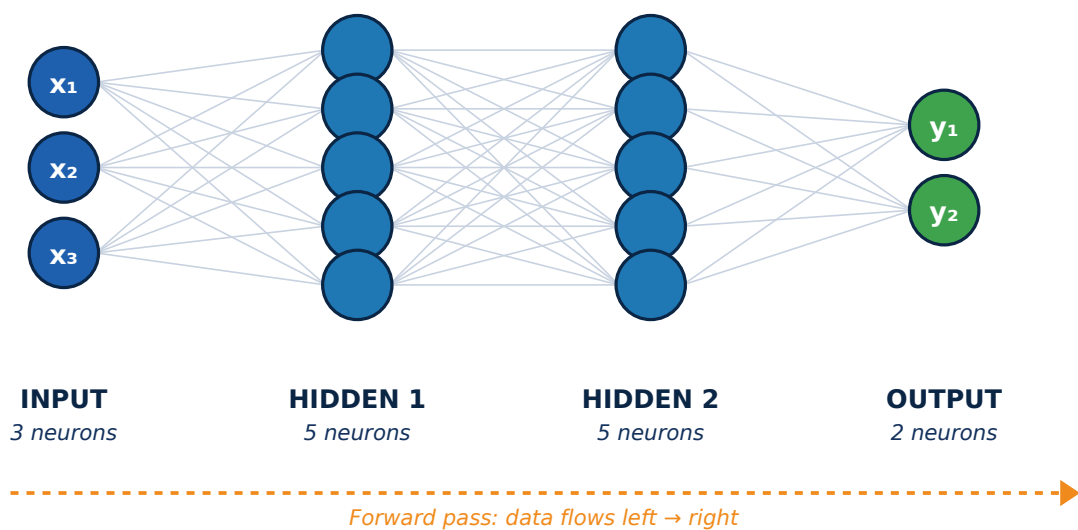


Figure 2.1 — A feedforward neural network with 3 inputs, two hidden layers, and 2 outputs

The three kinds of layers

Layer	Role
Input layer	Holds the raw features. One neuron per input value (pixel, word ID, sensor reading).
Hidden layers	Learn intermediate representations. "Deep" learning just means more than one hidden layer.
Output layer	Produces the prediction. Size depends on the task: 1 neuron for binary, N for N classes.

i INFO Universal Approximation Theorem

A feedforward network with just one hidden layer of enough neurons can approximate any continuous function to arbitrary accuracy. So why go deep? Because deep networks need far fewer neurons to learn the same function — and they learn hierarchical features (edges → shapes → objects in vision; characters → words → meaning in text).

3 Activation Functions

Without a non-linear activation function, a deep network is no more powerful than a single perceptron — a stack of linear operations collapses back into one linear operation. The activation function is what gives neural networks their expressive power. Three are essential to know.

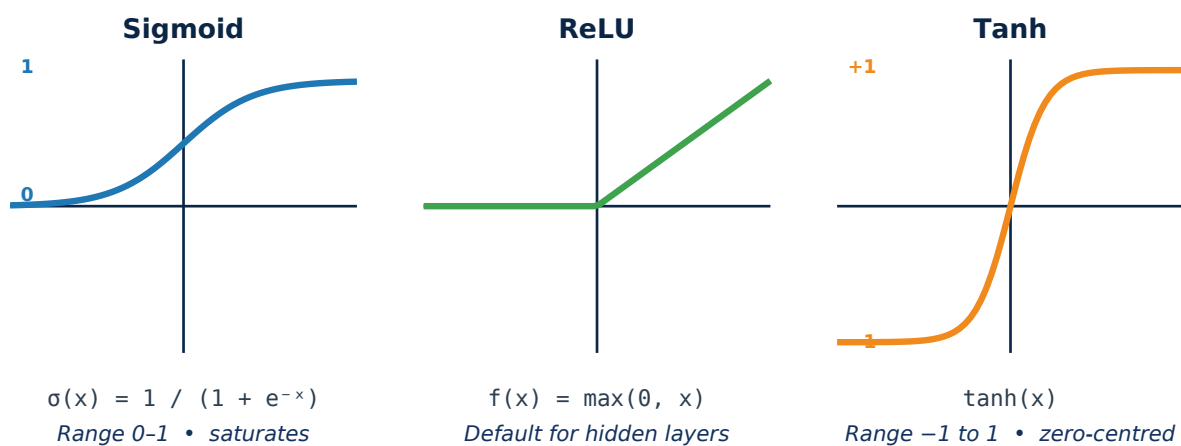


Figure 3.1 — The three classic activation functions: Sigmoid, ReLU, and Tanh

Activation	Use today	Watch out for
Sigmoid	Final layer for binary classification (output = probability).	Vanishing gradient — slow training in deep nets.
Tanh	Hidden layers in older RNNs; zero-centred output.	Still saturates; mostly replaced by ReLU.
ReLU	Default for all hidden layers in modern deep learning.	Dying ReLU — neurons stuck outputting 0.
Softmax	Final layer for multi-class classification (outputs sum to 1).	Numerically unstable — pair with log-loss.
GELU / SiLU	Used inside Transformers (BERT, GPT) — smooth ReLU variants.	Slightly slower than ReLU.

✓ TIP The 30-second decision guide

Hidden layers → ReLU (or GELU in Transformers). Binary output → Sigmoid. Multi-class output → Softmax. Regression output → no activation (linear). That covers 90% of all use cases.

4 How Neural Networks Learn

A freshly initialized network has random weights — its predictions are useless. Training is the iterative process of nudging those weights toward values that minimize a loss function, which measures how wrong the predictions are.

THE TRAINING LOOP

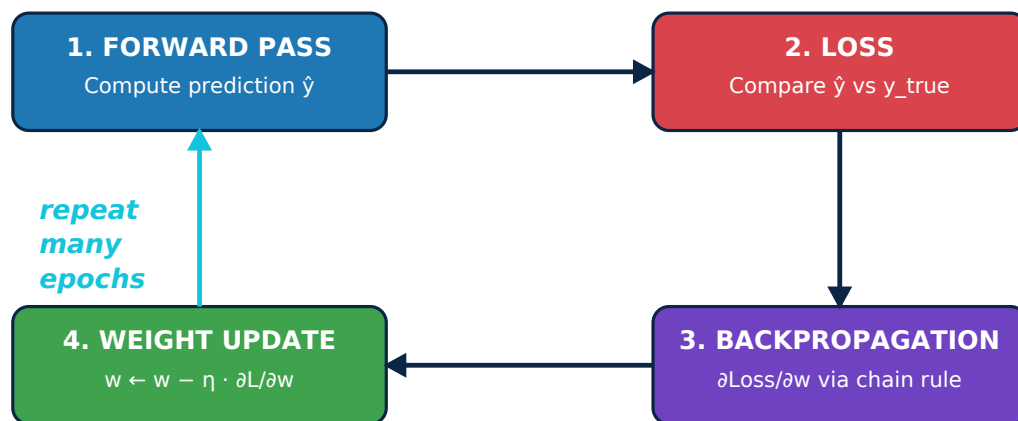


Figure 4.1 — The four-step training loop, repeated for thousands of epochs

Gradient descent — climbing down the loss hill

Once we know the loss, we use calculus to compute its gradient — the slope of loss with respect to every weight. The gradient tells us which direction makes loss increase; we go the opposite way. The size of the step is the learning rate η .

GRADIENT DESCENT

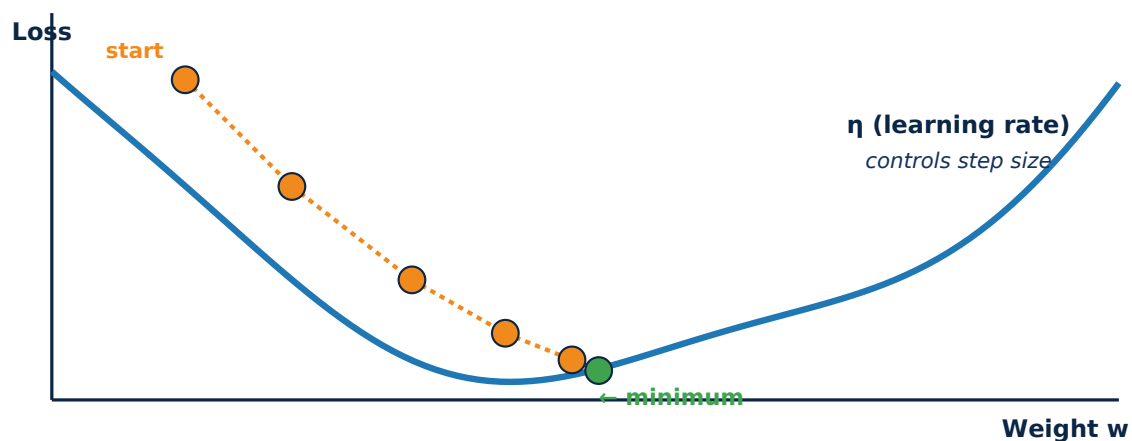


Figure 4.2 — Gradient descent walks downhill toward the minimum of the loss surface

! WARNING Learning rate — the most important hyperparameter

Too high: the ball overshoots and bounces around forever, loss diverges. Too low: the ball crawls and training takes weeks. Typical starting values are 0.001 for Adam optimizer, 0.01–0.1 for SGD. Tune this first, before anything else.

5 Practical Exercise

Time to put theory into practice. Work through this by hand first (paper & calculator), then verify in Python or on a spreadsheet.

Exercise 1 — Compute a perceptron's output

You are given a single neuron with three inputs:

- Inputs: $x_1 = 1.0$, $x_2 = 0.5$, $x_3 = -1.0$
- Weights: $w_1 = 0.4$, $w_2 = 0.6$, $w_3 = 0.2$
- Bias: $b = -0.3$
- Activation: ReLU, $f(z) = \max(0, z)$

Compute:

- the pre-activation $z = \sum w_i x_i + b$
- the post-activation $y = \text{ReLU}(z)$
- what happens if you change the bias to $b = +0.3$?

Solution

$$\begin{aligned} \text{(a) } z &= (0.4)(1.0) + (0.6)(0.5) + (0.2)(-1.0) + (-0.3) \\ &= 0.4 + 0.3 - 0.2 - 0.3 = 0.2 \end{aligned}$$

$$\text{(b) } y = \text{ReLU}(0.2) = \max(0, 0.2) = 0.2$$

(c) With $b = +0.3$, $z = 0.4 + 0.3 - 0.2 + 0.3 = 0.8 \rightarrow y = 0.8$. Raising the bias makes the neuron fire more strongly – it becomes "easier to activate." This is exactly what a higher y-intercept does to a line.

Exercise 2 — Build a 2-input perceptron in Python

Type this into any Python environment (no libraries required):

```
import math

def perceptron(x1, x2, w1, w2, b):
    z = w1*x1 + w2*x2 + b
    return max(0, z) # ReLU

# AND-gate weights (manually chosen)
print(perceptron(1, 1, 0.5, 0.5, -0.7)) # → 0.3 (fires)
print(perceptron(1, 0, 0.5, 0.5, -0.7)) # → 0.0 (off)
print(perceptron(0, 1, 0.5, 0.5, -0.7)) # → 0.0 (off)
print(perceptron(0, 0, 0.5, 0.5, -0.7)) # → 0.0 (off)
```

Congratulations — you have just implemented an AND gate as a neural network! Try changing the bias to -0.3 and see if you can make it an OR gate instead.



Self-Check Quiz

Five short questions to lock in today's concepts. Try first, then read the explanation.

1. Why do neural networks need non-linear activation functions?

Without non-linearity, every layer is a linear transformation, and a stack of linear transformations is itself linear. The whole network would collapse into one big perceptron and could only learn linearly separable patterns. Non-linearity is what gives deep networks their expressive power.

2. What does the bias term do in a neuron?

It shifts the activation function left or right along the input axis — exactly like the y-intercept of a straight line. Without bias, every neuron's decision boundary has to pass through the origin, which severely limits what it can learn.

3. What is one full pass through the training loop called, and what are its four steps?

One pass is an iteration; one full pass through the whole dataset is an epoch. The four steps are (1) forward pass — compute the prediction, (2) loss — measure error, (3) backpropagation — compute gradients of loss w.r.t. each weight, (4) update — nudge weights against the gradient.

4. Why is ReLU the default activation in modern deep networks?

It is cheap to compute (just $\max(0, x)$), doesn't saturate for positive inputs, and lets gradients flow back unchanged through any active neuron — solving the vanishing-gradient problem that plagued sigmoid/tanh in deep networks. The trade-off is the 'dying ReLU' issue, mitigated by variants like Leaky ReLU or GELU.

5. Your model's loss is decreasing, then suddenly jumps to infinity. What is the most likely cause?

The learning rate is too high — the optimizer overshoot the minimum and started climbing the loss surface. Reduce η by $10\times$ (e.g. from 0.01 to 0.001), or switch to an adaptive optimizer like Adam. This is the single most common training bug.

→ COMING UP TOMORROW Day 11 — Building Your First Neural Network in Python

Tomorrow we get hands-on: install PyTorch, build a 3-layer network from scratch, train it on the classic MNIST digits dataset, and watch the loss curve drop in real time. You will learn how to write a complete training script in under 50 lines of code. See you tomorrow — same time, same place. ayetechhub.com



Stay Connected With AYE Tech Hub

About this course

This is Day 10 of the AYE Tech Hub 30-Day AI Course — a beginner-to-intermediate curriculum that takes you from "what is AI?" to building, training, and deploying your own deep-learning models. Each day is a self-contained PDF with diagrams, worked examples, exercises, and a quiz. Catch up on any missed days at ayetechub.com.

About the author

Awet G. Nway is a Robotics, Automation and MEP Engineering specialist and the founder of AYE Tech Hub — an engineering education and services platform serving learners and businesses across Ethiopia and beyond.

Contact & resources

Website	www.ayetechub.com
Email	awetgknway@gmail.com
Social handles	@ayetechub @ayetechpro
Platforms	YouTube • Telegram • Instagram • LinkedIn • Threads • TikTok • Facebook
Services	Engineering education • PLC & Automation • MEP • HVAC • Solar • CAD/Revit • Interior & Architectural Design
Languages	English • Amharic • Tigrinya

Engineer the future. Learn it once. Apply it anywhere.

— AYE Tech Hub | 30-Day AI Course | Day 10 of 30 —